

搜索的疆域

搜索与回溯 · 8题 · C++ & Python 双语对照

NQ130 AcWing 842 AcWing 842

AcWing 842

输入 见原题

输出 见原题

范围 见原题

输入
见原题

输出
见原题

思路 见AcWing原题描述

技巧 参考模板库 CONTEXT.md

C++

```

#include <iostream>
using namespace std;

const int N = 10; //最大值
int n;           //读入的n
int path[N];     //记录路径每个位的值
bool used[N];    //记录第i个数是否被用过

void dfs(int u) {
    if (u == n) {
        //输出排列
        for (int i = 0; i < n; i++) cout << path[i]
<< " ";
        cout << endl;
        return;
    }
    //枚举数字1--n
    for (int i = 1; i <= n; i++) {
        if (!used[i]) {
            path[u] = i; //记录当前位path[u]的数字
            used[i] = true; //标记数字i为已经使用郭
            dfs(u + 1); //递归处理下一个位
            used[i] = false; //恢复现场
        }
    }
}

int main() {
    cin >> n;
    dfs(0);
    return 0;
}

```

Python

AcWing 842

NQ131 AcWing 843

AcWing 843

AcWing 843

输入 见原题

输出 见原题

范围 见原题

输入
见原题

输出
见原题

思路 见AcWing原题描述

技巧 参考模板库 CONTEXT.md

C++

```
#include <iostream>
using namespace std;

const int N = 20; //最大值
int n; //输入的n
char g[N][N]; //棋盘
//三个指示器列、对角线、反对角线,对角线以截距为编号
bool col[N], dg[N], udg[N];

void dfs(int u) {
    if (u == n) {
        //输出排列
        for (int i = 0; i < n; i++) puts(g[i]);
        puts("");
        return;
    }
    //枚举列0--n
    for (int i = 0; i < n; i++) {
        if (!col[i] && !dg[i + u] && !udg[i - u + n]) {
            g[u][i] = 'Q'; //当前位置设置为Q字符
            col[i] = dg[i + u] = udg[i - u + n] = true;
            //标记数字i为已经使用郭
            dfs(u + 1);
            //递归处理下一个位
            g[u][i] = '.';
            //恢复现场
            col[i] = dg[i + u] = udg[i - u + n] = false;
        }
    }
}

int main() {
    cin >> n;
    //初始化图
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++) g[i][j] = '.';
    dfs(0);
    return 0;
}
```

Python

AcWing 843

NQ132 AcWing 844 AcWing 844

AcWing 844

 见原题

 见原题

 见原题

输入

见原题

输出

见原题

思路 见AcWing原题描述

技巧 参考模板库 CONTEXT.md

C++

```

#include <algorithm>
#include <iostream>
#include <queue>
#include <cstring>
using namespace std;

typedef pair<int, int> PII; // Y总风格pair<int, int>声明

const int N = 107;

int n, m;
int g[N][N], d[N][N];

//广搜
int bfs() {

    queue<PII> q; //队列

    memset(d, -1, sizeof d);
    d[0][0] = 0;
    q.push({0, 0});

    // 四个方向向量
    int dx[] = {-1, 0, 1, 0}, dy[] = {0, 1, 0, -1};

    while (q.size())
    { //队列为空的时候退出广搜
        auto t = q.front(); //取队头
        q.pop(); //弹出队头

        //搜索四个方向
        for (int i = 0; i < 4; i++)
        {
            int x = t.first + dx[i], y = t.second + dy[i]; //待搜索的新坐标点

            if (x < 0 || x >= n || y < 0 || y >= m || g[x][y] != 0 || d[x][y] != -1) continue;

            d[x][y] = d[t.first][t.second] + 1;

            q.push({x, y}); //入队
        }
    }

    return d[n-1][m-1];
}

int main() {

    cin >> n >> m;

    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            cin >> g[i][j];

    cout << bfs() << endl;
}

```

Python

AcWing 844

```
return 0;  
}
```

NQ133 AcWing 845 AcWing 845

AcWing 845

输入 见原题

输出 见原题

范围 见原题

输入
见原题

输出
见原题

思路 见AcWing原题描述

技巧 参考模板库 CONTEXT.md

C++

```

#include <iostream>
#include <cstring>
#include <algorithm>
#include <unordered_map>
#include <queue>

using namespace std;

int bfs(string start)
{
    string end = "12345678x";

    queue<string> q;
    unordered_map<string,int> d;//字符串映射到整数

    q.push(start);
    d[start] = 0;//map的用法

    //方向向量
    int dx[4] = {1,-1,0,0},dy[4]={0,0,1,-1};

    //广搜
    while(q.size())
    {
        auto t = q.front();
        q.pop();

        int distance = d[t];
        if (t==end) return distance;//找到目标

        //查询x在字符串中的下标
        int k = t.find('x');
        int x = k / 3, y = k % 3;//转换为宫格坐标

        for (int i = 0; i < 4; i ++ )
        {
            int a = x + dx[i], b = y + dy[i];

            if (a >= 0 && a < 3 && b >= 0 && b < 3)
            {
                swap(t[k], t[a*3+b]);//移动x的位置
                if (!d.count(t))
                {
                    d[t] = distance + 1;
                    q.push(t);//压入队列
                }
                swap(t[k],t[a*3+b]);//还原现场
            }
        }
    }
    return -1;
}

int main()
{
    string c, start;
    //去掉空格
    for (int i = 0; i < 9; i ++ )
    {
        cin>>c;
        start += c;
    }
}

```

Python

AcWing 845

```
cout<<bfs(start)<<endl;  
}
```

NQ134 AcWing 846

AcWing 846

AcWing 846

输入 见原题**输出** 见原题**范围** 见原题

输入

见原题

输出

见原题

思路 见AcWing原题描述**技巧** 参考模板库 CONTEXT.md

C++

```

#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 100007, M = N*2; // 无向图

int n;
int h[N], e[M], ne[M], idx;
int ans = N;
bool st[N]; // 顶点n是否访问

void add(int a, int b) // 添加一条边a->b
{
    e[idx] = b, ne[idx] = h[a], h[a] = idx ++ ;
}

int dfs(int u)
{
    st[u] = true;

    int size = 0; // 去掉i节点后, i子树的最大子树节点数
    int sum = 0; // i子树的节点总数
    for (int i = h[u]; i != -1; i = ne[i])
    {
        int j = e[i];
        if (st[j]) continue;

        int s = dfs(j); // 返回j子树的大小
        size = max(size, s);
        sum += s; // 计算所有子树和
    }

    size = max(size, n - sum - 1); // i子树和与 非i子树部分的和, 取最大值
    ans = min(ans, size);

    return sum+1; // 加上i节点本身
}

int main()
{
    cin >> n;
    memset(h, -1, sizeof h);
    for (int i = 0; i < n-1; i ++ )
    {
        int a, b;
        cin >> a >> b;
        add(a, b), add(b, a); // 无向边
    }
    dfs(1);
    cout << ans << endl;
    return 0;
}

```

Python

AcWing 846

NQ135 AcWing 847 AcWing 847

AcWing 847

- 输入 见原题
- 输出 见原题
- 范围 见原题

输入	输出
见原题	见原题

思路 见AcWing原题描述

技巧 参考模板库 CONTEXT.md

C++

```

#include <iostream>
#include <cstring>
#include <algorithm>
#include <queue>

using namespace std;

const int N = 100007; // 有向图

int n, m;
int h[N], e[N], ne[N], idx;
int d[N];

void add(int a, int b) // 添加一条边 a->b
{
    e[idx] = b, ne[idx] = h[a], h[a] = idx ++ ;
}

int bfs()
{
    memset(d, -1, sizeof d);
    queue<int> q;
    d[1] = 0; // 起始点
    q.push(1);

    while (q.size())
    {
        auto t = q.front();
        q.pop();

        for (int i = h[t]; i != -1; i = ne[i])
        {
            int j = e[i];
            if (d[j] == -1)
            {
                d[j] = d[t] + 1; // 层次加1
                q.push(j);
            }
        }
    }
    return d[n];
}

int main()
{
    cin >> n >> m;
    memset(h, -1, sizeof h);

    for (int i = 0; i < m; i++)
    {
        int a, b;
        cin >> a >> b;
        add(a, b);
    }

    cout << bfs() << endl;
    return 0;
}

```

Python

AcWing 847

AcWing 777

输入 见原题**输出** 见原题**范围** 见原题

输入

见原题

输出

见原题

思路 见AcWing原题描述**技巧** 参考模板库 CONTEXT.md**C++**

```
#include <iostream>

using namespace std;

int main()
{
    string str;

    while (cin >> str, str != ".")
    {
        int len = str.size();

        for (int n = len; n; n -- )
            if (len % n == 0)
            {
                int m = len / n;
                string s = str.substr(0, m);
                string r;
                for (int i = 0; i < n; i ++ ) r +=
s;

                if (r == str)
                {
                    cout << n << endl;
                    break;
                }
            }

        return 0;
    }
}
```

Python

AcWing 777

NQ137 AcWing 778 AcWing 778

AcWing 778

输入 见原题

输出

见原题

范围

见原题

输入	输出
见原题	见原题

思路 见AcWing原题描述

技巧 参考模板库 CONTEXT.md

C++

```

#include <iostream>

using namespace std;

int main()
{
    string s, s1, s2;

    char c;
    while (cin >> c, c != ',') s += c;
    while (cin >> c, c != ',') s1 += c;
    while (cin >> c) s2 += c;

    if (s.size() < s1.size() || s.size() < s2.size()) puts("-1");
    else
    {
        int l = 0;
        while (l + s1.size() <= s.size())
        {
            int k = 0;
            while (k < s1.size())
            {
                if (s[l + k] != s1[k]) break;
                k ++ ;
            }
            if (k == s1.size()) break;
            l ++ ;
        }

        int r = s.size() - s2.size();
        while (r >= 0)
        {
            int k = 0;
            while (k < s2.size())
            {
                if (s[r + k] != s2[k]) break;
                k ++ ;
            }
            if (k == s2.size()) break;
            r -- ;
        }

        l += s1.size() - 1;

        if (l >= r) puts("-1");
        else printf("%d\n", r - l - 1);
    }

    return 0;
}

```

Python

AcWing 778